

INSuRE Spring 2023

Final Project Report

Cryptographic Protocol Analysis and Verification

Sponsor: National Security Agency

Technical Directors:

Ed Zieglar (evziegl@uwe.nsa.gov)

Mentor:

Enis Golaszewski (golaszewski@umbc.edu)

The University of Texas at Dallas

Team 1

Manasa Kandala (mxk200008@utdallas.edu)

Pragathi Vemulapalli (pxv200020@utdallas.edu)

Rahul Muthyam (rxm200088@utdallas.edu) (Team Lead)

Saloni Shivdasani (sxs170015@utdallas.edu)

Shreyans Patel (ssp210009@utdallas.edu)

Faculty

Brian Ricks (bwr031000@utdallas.edu)

Table of Contents

PROJECT SUMMARY.....	3
TOP LEVEL PROJECT DESCRIPTION.....	3
KEYWORDS	3
ACRONYMS.....	3
TEAM MEMBER RESPONSIBILITIES	4
<i>Manasa Kandala</i>	4
<i>Pragathi Vemulapalli</i>	4
<i>Rahul Muthyam</i>	4
<i>Saloni Shivdasani</i>	4
<i>Shreyans Patel</i>	5
ESTIMATED PROJECT BUDGET	5
EXECUTIVE SUMMARY.....	5
MOTIVATION.....	6
LITERATURE REVIEW.....	7
SPECIFIC AIMS.....	8
PROJECT PLAN AND APPROACH.....	8
PROJECT PLAN.....	8
PROJECT SCHEDULE.....	9
RESOURCES REQUIRED AND BUDGET.....	11
DELIVERABLES	11
ANTICIPATED ISSUES AND CHALLENGES.....	12
BROADER IMPACT.....	12
MIDTERM PROGRESS.....	13
FINDINGS.....	13
<i>About CPSA</i>	13
<i>About G NAP</i>	14
MODELING G NAP USER-CODE INTERACTION FOR CPSA	15
CHALLENGES FACED	16
NEXT STEPS	17
FINAL PROGRESS.....	17
SOFTWARE-ONLY AUTHORIZATION SEQUENCE	17
<i>Request Access (1)</i>	18
<i>Grant Access (2)</i>	19
<i>Access API (3)</i>	19
<i>API Response (4)</i>	20
<i>Results – Client Instance Perspective Shape</i>	20
<i>Results - Authorization Server Perspective Shape</i>	21
<i>Results - Resource Server Perspective Shape</i>	22
USER-CODE INTERACTION SEQUENCE.....	22
<i>Request Access (1)</i>	25
<i>Interaction Needed (2)</i>	25
<i>Open URI (4), AuthN (5), Code (6), AuthZ (7) and Completed (8)</i>	26
<i>Continue Request (B) (11)</i>	26
<i>Grant Access (12)</i>	26
<i>Access API (13)</i>	27
<i>API Response (14)</i>	27

<i>Results – Client Instance Perspective Shape</i>	28
<i>Results – Client Instance Perspective (Unrealized)</i>	29
<i>Results - Authorization Server Perspective Shape</i>	30
<i>Results - Resource Server Perspective Shape</i>	31
NEXT STEPS	31
CONCLUSION.....	32
TEAM MEMBER BIOGRAPHIES	32
<i>Manasa Kandala</i>	32
<i>Pragathi Vemulapalli</i>	33
<i>Rahul Muthyam</i>	33
<i>Saloni Shivdasani</i>	33
<i>Shreyans Patel</i>	34
BIBLIOGRAPHY	34

Project Summary

Top Level Project Description

The security of cryptographic protocols, particularly those used in computer networks to exchange sensitive information, is very important. Many protocols (especially the ones designed in the late 1900s) have vulnerabilities due to design decisions made during their development, and because these protocols are often used in highly sensitive applications, it is crucial to thoroughly evaluate their security properties and make any necessary amendments. For this purpose, various tools have been developed such as the Cryptographic Protocol Shapes Analyzer (CPSA) which outputs sets of shapes describing all possible protocol executions and Maude-NPA (NRL Protocol Analyzer) which works in a backward fashion concerning CPSA [1].

The Grant Negotiation and Authorization Protocol (GNAP) is being developed as the next-generation standard for user authorization, to replace OAuth2 [2]. GNAP improves upon various aspects of OAuth2 and aims to provide a secure and seamless experience for all parties involved. For instance, the initial design of the OAuth2 protocol was primarily intended for use in a web browser, and support for mobile applications was added later on, which further complicated an already complex protocol [3]. It is very important to validate the security properties of the GNAP protocol before it becomes a standard for user authorization, as any vulnerabilities discovered after its widespread adoption could have significant consequences in the computing domain, like that of the Log4j vulnerability.

In this project, we aim to study GNAP and onboard to the CPSA tool so that we can analyze the protocols used in the authentication process between the end-user and the AS to find out any vulnerabilities that might exist.

Keywords

Cryptographic, Protocol, Analysis, Verification, GNAP, CPSA, Network, Security

Acronyms

1. GNAP: Grant Negotiation and Authorization Protocol
2. CPSA: Cryptographic Protocol Shapes Analyzer

3. CSRF: Cross-Site Request Forgery
4. AS: Authorization Server
5. RO: Resource Owner
6. 2FA: Two-Factor Authentication

Team Member Responsibilities

Manasa Kandala

With her work experience and specialization in Information Assurance Track, Manasa will be addressing cybersecurity and network security-related learning curves and ensuring everyone in the team understands all the underlying concepts for the project. She will also help with the research on GNAP to identify the protocols that can be analyzed in the authentication process of GNAP.

Pragathi Vemulapalli

Pragathi with her knowledge of Web Programming Languages will leverage it to study various kinds of requests that are made in the background in GNAP. She will analyze these to find possible attacks that can be done to compromise the communication between the legitimate end-user and AS. She will use her programming abilities to address coding-related matters while working on the project.

Rahul Muthyam

Being the team lead, Rahul will ensure the team is working efficiently and all targets and deadlines are met. He will serve as the middleman between the team and the various stakeholders like the technical director, faculty, and others. He will work as a primary researcher to find and generate resources for the team for reference and understanding. He will also be responsible for ensuring all meetings with the technical director and faculty are carried out regularly and everyone stays in the loop.

Saloni Shivdasani

Saloni will be utilizing her software engineering experience to work on tasks requiring coding. She will be analyzing the code that is involved in GNAP for authentication and

understanding its bounds with the communication channel. She will also serve as a researcher for the project to identify relevant studies and share them with the team.

Shreyans Patel

Shreyans will be using his C/C++ coding skills to understand codes involved in server-client communication. He will also address any programming-related learning curves and ensure all coding-demanding tasks are completed on time. He will also carry out all team lead responsibilities in Rahul's absence in unprecedented situations.

Estimated Project Budget

The team plans to spend about 530 hours or about 8 hours per week per person for the semester and we do not expect to incur any monetary costs.

Executive Summary

Cryptographic protocols in use today were created in the late 1900s and were not thoroughly examined at the time due to a lack of resources. As protocols become more complex, it becomes harder to examine all possible execution scenarios and identify weaknesses in them. Despite this glaring issue, these protocols have since become widely adopted standards.

One such example of this is OpenSSL. It is a popular cryptographic framework used to build robust commercial-grade applications and because of the way it was implemented, a zero-day vulnerability, "Heart-bleed" was discovered. This vulnerability allows a remote attacker to access sensitive information, such as user credentials and secret keys, due to incorrect memory management in the TLS heartbeat extension. We require a reliable method for analyzing cryptographic protocols to identify vulnerabilities at an early stage. This is where tools like CPSA come into the picture.

The Cryptographic Protocol Shapes Analyzer (CPSA) is a tool that evaluates whether a cryptographic protocol satisfies its security properties. It endeavors to list all unique executions, known as shapes, of a given cryptographic protocol by utilizing a simple and verifiable input that outlines the protocol's variables, participant roles, and exchanged messages. The CPSA program then examines initial behavior to determine the shapes that are consistent with it.

Unlike other tools, CPSA's objective is to provide a comprehensive portrayal of all possible executions, regardless of any specific security property, making it easy to identify authentication and confidentiality properties, attacks, and anomalies. In this project, we will leverage this tool to analyze a potential replacement for the OAuth2 protocol, popularly known as GNAP. While it is in the early stages of development, it can be beneficial to analyze and find out any structural vulnerabilities in this protocol at this stage itself before it gets more complex.

The Grant Negotiation and Authorization Protocol (GNAP) is a protocol designed for secure and efficient access to protected resources. It outlines trust relationships between software components and roles, such as end-user and client, etc. GNAP also defines grant requests, grant responses, and interaction modes to aid the authorization and negotiation process. GNAP prioritizes the privacy of end users and resource owners by using cryptographic keys and signatures to associate access tokens with the client instance and using hash calculations to secure query parameters added to the URI.

In addition to that, GNAP provides mechanisms for the client instance to prove key possession and for the AS to verify that the same client instance is making follow-up requests for a given grant request. In summary, GNAP offers an easy, secure, and efficient way to access protected resources while maintaining end-user and resource-owner privacy.

All in all, considering that GNAP is in the development phase, it is a potential choice for formal analysis. We will use CPSA to analyze the authentication properties of GNAP and attempt to find any vulnerabilities in its structure. Any discoveries on this matter would benefit globally standardizing GNAP as a secure protocol.

Motivation

An essential element in assuring the security of communication networks is the analysis and verification of cryptographic protocols. Standards developed by groups like the IETF are used to build protocols, allowing for a great deal of customization. The protocol as outlined in the RFC must have three independent and cooperative implementations to become a standard. Although the protocol's functionality is ensured, its security is not. A whiteboard implementation of the protocol is developed together with a threat modeling study to find possible attack vectors to reduce security

concerns. Nevertheless, employing manual techniques still leaves room for error and potential security issues, even with experience. Formal analysis is useful in this situation. By using formal methods, as many potential attacks as feasible can be removed, hence the protocol may be made more safe and ready for standardization.

Literature Review

The Grant Negotiation and Authorization Protocol (GNAP), a next-generation authorization protocol, is being developed as a replacement for OAuth2 to improve security and usability. Despite being in its developmental stage and yet to be established as a standard, a limited number of studies have analyzed the security properties of GNAP.

The "Security Analysis of Attack Surfaces on the Grant Negotiation and Authorization Protocol" thesis paper from the Department of Computer Science and Engineering, Chalmers University of Technology, and the University of Gothenburg, published in 2021, aimed to examine the GNAP protocol in light of the common attack surfaces of OAuth2.0 such as Redirect URI attacks, Access code hijacking, CSRF, and AS mix-up attacks. The study evaluated the robustness of GNAP against these attacks and compared the results with OAuth2.0 and OAuth2.1 protocols. The findings showed that GNAP's cryptographic design effectively prevents attacks that involve data manipulation or extra input. However, AS mix-up attacks may still be feasible in a specific scenario due to the less secure client aspect of the protocol [4].

Another analysis, the "Security Analysis of the Grant Negotiation and Authorization" thesis paper from the Institute of Information Security, University of Stuttgart, focused on modeling GNAP within the Web Infrastructure Model and defining several security properties for the protocol. This study tested the security properties of GNAP against various vulnerabilities found in OAuth2.0, OpenID, and other related protocols. The analysis verified some of the security properties of GNAP and discovered three attacks on the protocol, namely 307 Redirect Attack, Cuckoo Token Attack, and the Client Instance Mix-Up Attack. The researchers worked with the GNAP editors to resolve these issues and incorporate the fixes into the protocol.

Specific Aims

The team is set on analyzing various sequences of the GNAP protocol by using CPSA, with a focus on the authentication parts of the sequences. We are particularly interested in the User-Code Interaction sequence, as it involves 2FA when the Resource Owner (RO) is authenticating the authorization request with the Authorizing Server (AS). These kinds of interactions are crucial as they could lead to vulnerabilities if not implemented correctly. The team plans to start with this sequence and if time permits, analyze similar vulnerabilities in other sequences of the protocol. At the end of the analysis, the team will generate a formal report of their findings, which can be used as a basis for further analyses of the protocol.

Project Plan and Approach

Project Plan

Our tentative research project plan uses a three-phased approach. During Phase One of the project, the focus will be on conducting extensive research on the GNAP protocol to gain a thorough understanding of it. Additionally, the team members will receive training in the use of the CPSA tool so they can effectively utilize it in the analysis of the algorithm. The goal of this phase is to equip the team with the necessary knowledge and skills to successfully carry out the rest of the project.

The second phase of the project involves a detailed analysis of various sequences of the GNAP protocol. The focus will be on the authentication sub-parts of the particular sequence, where the team will use the CPSA tool to identify any potential vulnerabilities. If any vulnerabilities are discovered, the team will then examine if they exist in other sequences of the GNAP protocol as well. The objective of this phase is to ensure the security and reliability of the GNAP protocol by identifying and addressing any weaknesses or threats.

The third and final phase of the project will involve the refinement and finalization of the findings from the previous phases. The team will focus on polishing their results and getting the deliverables organized and in a presentable format. A comprehensive final report will be generated that includes a summary of the research conducted, the findings, and any recommendations for improving the

GNAP protocol. The objective of this phase is to present the results of the project clearly and concisely, highlighting the significance of the work carried out.

- Phase One - Researching GNAP and getting acquainted with the CPSA tool.
- Phase Two - Using the CPSA tool on GNAP to analyze the authentication algorithm.
- Phase Three - Finalizing our findings and generating a final report.

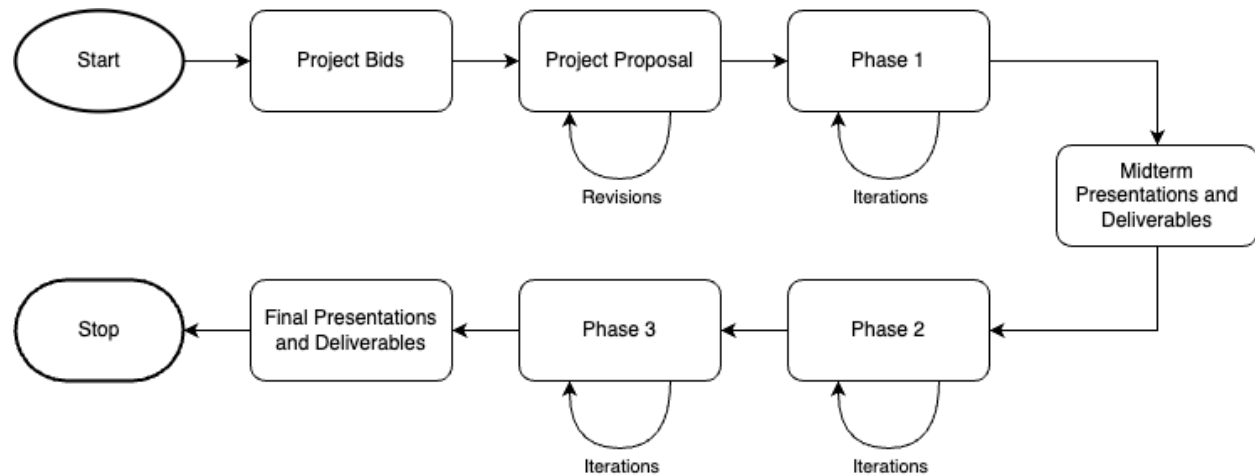


Figure 1: Project Schedule Flowchart

Project Schedule

The schedule below showcases deliverables/milestones with their due dates as well as the expected due dates. The expected due date is designed to allow the team time for peer review to confirm all important information has been explained appropriately.

Table 1: Project Key Deliverables Schedule

Sr. No.	Key Deliverables	Due Date / Expected Completion Date
1.	Bids	January 24, 2023
2.	Project Proposal	February 6, 2023
3.	Midterm Presentation Draft	March 3, 2023

4.	Midterm Presentation Submission	March 23, 2023
5.	Midterm Presentation	March 24, 2023
6.	Midterm Presentation Deliverables	March 24, 2023
7.	Draft Final Presentation	April 28, 2023
8.	Final Presentation Submission	May 4, 2023
9.	Final Presentation	May 5, 2023
10.	Final Presentation Deliverables	May 5, 2023

Table 2: Project Phase Completion Schedule

Sr. No.	Phase	Tentative Completion Date Range
1.	Phase One Researching GNAP and getting acquainted with the CPSA tool	February 27, 2022 - March 4, 2022
2.	Phase Two Using the CPSA tool on GNAP to analyze the authentication algorithm	March 14, 2022 - March 21, 2022
3.	Phase Three Finalizing our findings and generating a final report	April 3, 2022 - April 10, 2022

Resources Required and Budget

In terms of time, the total project budget for the team is estimated at 520 hours. This is based on the team working 8 hours per week per member for the 13 weeks, centered on the final report due date. Additionally, the team accounts for another 6 hours of sponsor consultation time and about 4 hours of sponsor virtual text communication time. Therefore, the total team time budget is 520 hours, and the sponsor time budget is approximately 10 hours. Thus, the total time budget is 530 hours.

As our team does not anticipate the use of any specialized equipment or premium software, we do not have any monetary budget.

Table 3: Required Resources concerning Time and Monetary Costs

Sr. No.	Resource Name	Time Cost (Hours)	Dollar Cost (USD)	Mandatory (Yes/No)
1.	Team R&D	520	0	Yes
2.	Sponsor Consultation time	6	0	Yes
3.	Sponsor Virtual communication	4	0	Yes
4.	Protocol Verification Tools (CPSA)	N/A	0	Yes
5.	Totals	530	0	N/A

Deliverables

Table 4: Deliverables and their due dates

Sr. No.	Key Deliverables	Due Date / Expected Completion Date
1.	Bids	January 24, 2023

2.	Project Proposal	February 6, 2023
3.	Midterm Presentation	March 24, 2023
4.	Midterm Report	March 24, 2023
5.	Final Presentation	May 4, 2023
6.	Final Report	May 5, 2023

Anticipated Issues and Challenges

The main challenge we anticipate at this time is the lack of previous research on protocol analysis and verification techniques. With a subject matter that is relatively unexplored and highly advanced, we anticipate a slight learning curve for the team. Considering how open-ended this topic is, there is a high chance that our vision could deviate from our project plan.

Broader Impact

The broader impacts of our project are made evident by the motivation and specific aims. The team's final work investigates aspects of cryptographic protocol analysis and verification that have conventionally received little attention. The team's ultimate goal is to advance a new line of thinking about protocol analysis. In particular, the finished output will draw attention to potential avenues for verifying the protocol's security properties or identifying security flaws so that the security proof may start to be standardized.

Midterm Progress

Findings

About CPSA

We have talked about CPSA in previous sections of our report. However, in this section, we talk about how our knowledge of CPSA improved after working on the project. We got a better understanding of why CPSA is the tool to use for our analysis. We learnt how we can model protocols in CPSA through the training given by our mentor Enis and completing an exercise on the Needham–Schroeder protocol’s analysis using CPSA.

The Cryptographic Protocol Shapes Analyzer (CPSA) is a software tool developed by MITRE to assist in the analysis of cryptographic protocols and it is being used since 2016. Given a scenario, it attempts to find all the possible executions for a cryptographic protocol (that are known as shapes in CPSA terminology) by assuming the whole network to be compromised. For example, the adversary can add/modify/flush any message that flows through the network. CPSA generates different shapes in which participants can communicate on the network and we can analyze these shapes to find any security flaws.

Figure 2 shows an example of a protocol where there is a security concern. In this example, we have our three famous actors on a network - Alice, Bob, and Charlie. There can be many sequences in which these three actors can communicate with each other, and this sequence is one of them. Here, Alice sends a message containing her name and nonce encrypted with Bob’s public key. Bob extracts the message contents and encrypts them again with Charlie’s public key and forwards it to him. Now Charlie decrypts this message and sees that Alice is trying to communicate with him. So, he extracts Alice’s nonce from the message, adds his own nonce and encrypts it under Alice’s public key and sends it back to Alice. Bob intercepts this message and simply forwards it to Alice as it is. When Alice receives this message, she extracts her nonce and determines this new nonce is from Bob but in reality it is from Charlie. To complete the handshake, she extracts the nonce she received and encrypts it with Bob’s public key sending it to him. When Bob receives this message, he extracts Charlie’s nonce and encrypts it under Charlie’s public key and sends it back to him.

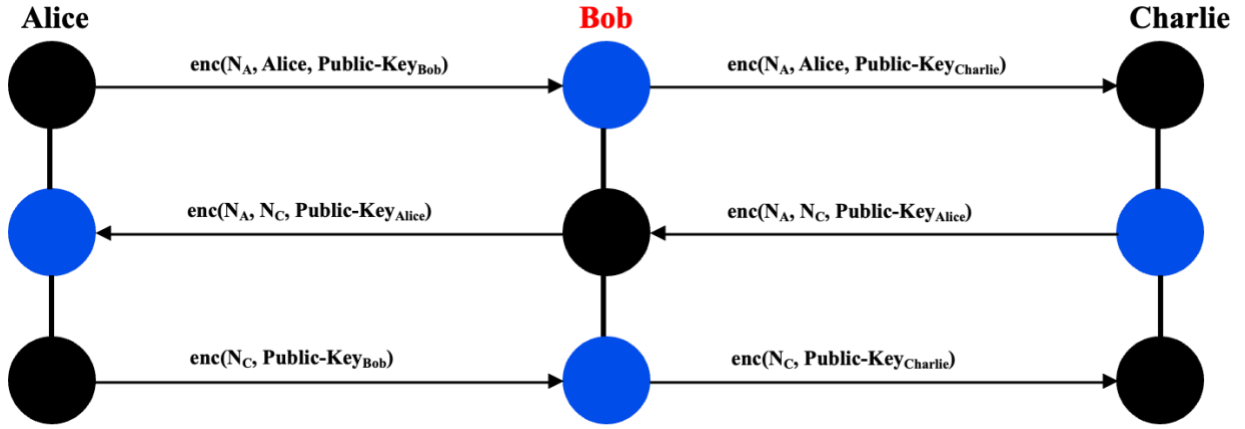


Figure 2: Example of a protocol with security concerns that can be found using CPSA

So now this is a big problem that Alice thinks she is talking to Bob and Charlie thinks he is talking to Alice. Also, Bob was able to read Charlie's nonce. In future communications, Bob will be able to read all messages written by Alice which are exclusively for Charlie.

Hence, since we cannot guarantee who created the message, this sequence can be used by an attacker which means this protocol is not secure. Since there is no easy way of manually coming up with such sequences in large protocols, CPSA becomes a very useful tool.

About GNAP

In our initial research on GNAP, we tried to understand multiple sequences to find the ones that are commonly used and tend to insecure. We looked into User-Code Interaction, Asynchronous Authorization, Cross-User Authentication and so on. From our study and discussion with our technical director and mentor, we have decided to start with the User-Code Interaction.

Modeling GNAP User-Code Interaction for CPSA

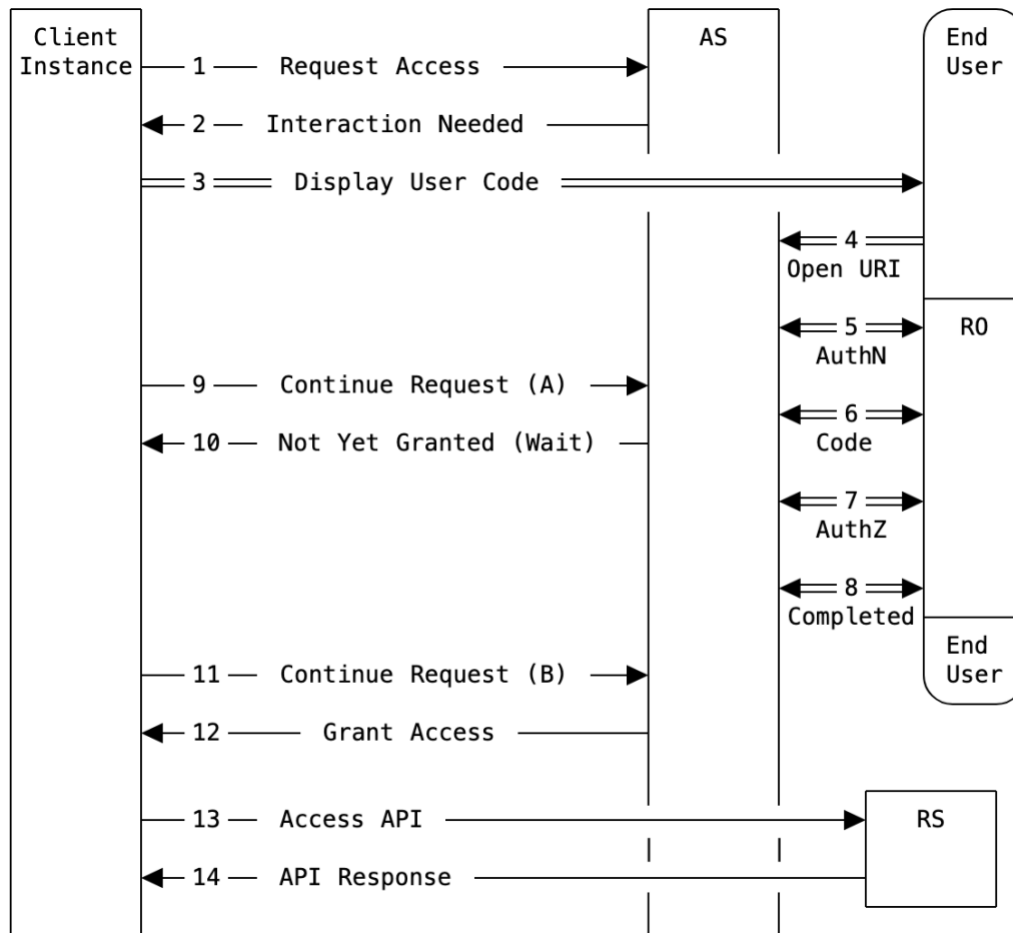


Figure 3: User-Code Interaction (GNAP)

Figure 3 shows the User-Code Interaction sequence of GNAP. In this sequence we have the following actors:

- (AS) Authorization Server: The brains of the system.
- (RS) Resource Server: A server hosting a resource.
- (RO) Resource Owner: An entity owning a resource on the Resource Server.
- End User: The end user of the system (usually a human being).
- Client Instance: A piece of software (web browser, mobile application, etc.) used for interactions by the End User.

GNAP has its own set of assumptions, and they are:

1. The underlying communication channels are secure. Therefore, TLS can be used to secure communication between any two entities.
2. No party is compromised i.e., all actors involved in the protocol are assumed to be honest.
3. The client requests are usually signed but the server responses are not. This means it is assumed that in a real environment (such as HTTPS) the client will automatically verify the identity of the server. This is done during the setup phase of the connection by verifying the certificate provided by the AS with the Certificate Authority.

Using these assumptions, we are currently working on understanding all these 14 interactions to filter out the parameters we need to use while modeling this sequence for CPSA. This is the part that our team is currently working on. Table 5 shows our progress in filtering out the said parameters which we would need to start modeling User-Code Interaction.

Table 5: Filtered parameters to model User-Code Interaction for analysis in CPSA

Sr. No.	(Interaction Number) Interaction	Interaction Name	Important Parameters for Modeling
1.	(1) CI $\rightarrow$ AS	Request Access	access (name), client (name), client_key (non-origin), msg_signature (...)
2.	(2) AS $\rightarrow$ CI	Interaction Needed	user_code (uniq-origin), continue_access_token (uniq-origin)
3.	(3) CI $\rightarrow$ EU	Display User Code	user_code
4.	(4) EU $\rightarrow$ AS	Open URI	[NA]
5.	(9) CI $\rightarrow$ AS	Continue Request (A)	continue_access_token, msg_signature (...)
6.	(10) AS $\rightarrow$ CI	Not Yet Granted (Wait)	continue_access_token (uniq-origin - if AS wants to rotate) (OPTIONAL)
7.	(5) AS $\leftrightarrow$ RO	AuthN	Long-Term-Keys (LTK)
8.	(6.1) RO $\rightarrow$ AS	Code	user_code
9.	(6.2) AS $\rightarrow$ RO	Code	random_data (uniq-orig)
10.	(7) AS $\leftrightarrow$ RO	AuthZ	[NA]
11.	(8) AS $\leftrightarrow$ RO	Completed	[NA]
12.	(11) CI $\rightarrow$ AS	Continue Request (B)	continue_access_token, msg_signature (...)
13.	(12) AS $\rightarrow$ CI	Grant Access	access_token (uniq-orig)
14.	(13) CI $\rightarrow$ RS	Access API	access_token, msg_signature (...)
15.	(14) RS $\rightarrow$ CI	API Response	data (uniq-origin)

Challenges Faced

We observed some delays in getting our CPSA training started due to which we had to push some targets back. However, we have been putting in extra work to handle this minor setback and we are positive we can still meet the original set deadlines. We received a lot of support from our

Technical Director Dr. Edward Zeiglar and mentor Enis Golaszewski to make sure we have all that we need to move ahead with a high velocity.

Next Steps

We are currently working on modeling one sequence of GNAP for analyzing with CPSA and we aim to finish this model along with adding TLS macros and using CPSA features like LTKs or Long-Term-Key and channels. Once we do that, we will come up with different scenarios to analyze, generate graphs and verify the security properties of each scenario. If we see any vulnerabilities, we will try to find a solution to fix them. If time permits, we will also model other GNAP sequences for analysis.

Final Progress

In order to model the GNAP protocol, we identified several sequences that are commonly used and vulnerable to attacks. These sequences included User-code interaction, Asynchronous Authorization, Cross-User Authorization, and Software-only Authorization. Initially, our plan was to focus on the first three sequences; however, we eventually became more interested in the Software-only Authorization sequence.

The reason for this is because this particular sequence does not involve an end-user. Therefore, we decided to prioritize the modeling of the Software-only Authorization sequence before moving on to the User-Code Interaction sequence.

Software-only Authorization Sequence

The Software-only Authorization sequence is shown in Figure 4. Unlike other GNAP sequences, the Software-only Authorization sequence uses MTLS (Mutual Transport Layer Security) in all of its interactions. This means that both the client and the server authenticate each other by validating their TLS certificates with the Certificate Authority (CA).

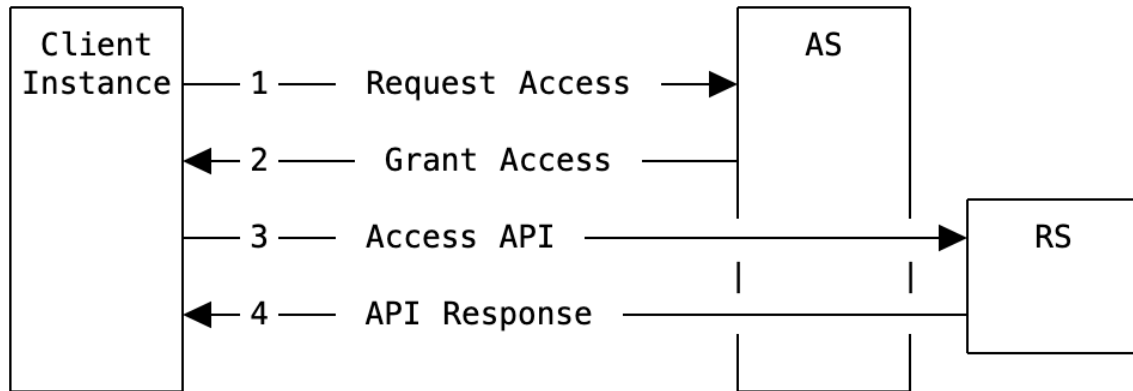


Figure 4: Software-only Authorization (GNAP)

The Software-only Authorization sequence allows a client instance to request access to a resource without the need for a resource owner to be involved at runtime to approve the decision. Therefore, the client instance does not interact with a resource owner. The steps involved in this sequence are:

1. The client instance requests access to the resource without sending any interaction modes to the server.
2. The authorization server grants access to the resource in the form of access tokens to the client instance based on its identity and requested access. Since there is no user involved, no access to the resource owner's information is required.
3. The client instance uses the access token to call the resource owner.
4. The resource owner validates the access token and responds appropriately to the API.

This sequence demonstrates how access can be granted to a client instance without requiring a resource owner to approve the decision at runtime.

Request Access (1)

For our model of the User-Code Interaction sequence, we identified three key parameters that must be included in the interaction:

1. Client Key: This parameter is necessary for the server to identify the client making the request.

2. Resource Server ID: This parameter identifies the specific resource that the client is requesting access to.
3. Resource Server Permission: This parameter indicates the level of access that the client requires to access the resource on the resource server.

These three parameters are critical for the successful execution of the User-Code Interaction sequence in our model.

Grant Access (2)

For the Software-only Authorization sequence, the authorization server takes the parameters sent by the client and creates an access token that is bound to those parameters. This can be achieved using JWTs or JSON web tokens, where all the required fields that identify the access token request are embedded, including:

1. Client Instance Key
2. Resource Server ID
3. Resource Server Permission

The authorization server signs this token with its private key, which makes the message tamper-proof. To model this in CPSA, we use the inverse of the public key (shown by `invk`) to obtain the private key. By doing so, we can ensure that the token has been properly signed and that the parameters sent by the client are valid.

Access API (3)

To access the API in the Software-only Authorization sequence, the client uses the access token received from the authorization server to proceed with its access request for the resources on the resource server. This access token cannot be modified since it was signed by the authorization server initially using its private key before sending it to the client. This ensures that the token is secure and cannot be tampered with during transit.

API Response (4)

In our access control model, the resource owner verifies the access token based on the type of request made by the client, ensuring that both the client and the token possess sufficient permissions to execute the requested access. Once this verification process is complete, the resource owner returns the necessary data along with the data hash and access token binding to associate this response with the request that utilized the access token.

Our model leverages CPSA4 channels to replicate the characteristics of TLS. These channels offer a combination of confidentiality and authenticity for the transmission of data, ensuring secure and reliable communication between parties.

Results – Client Instance Perspective Shape

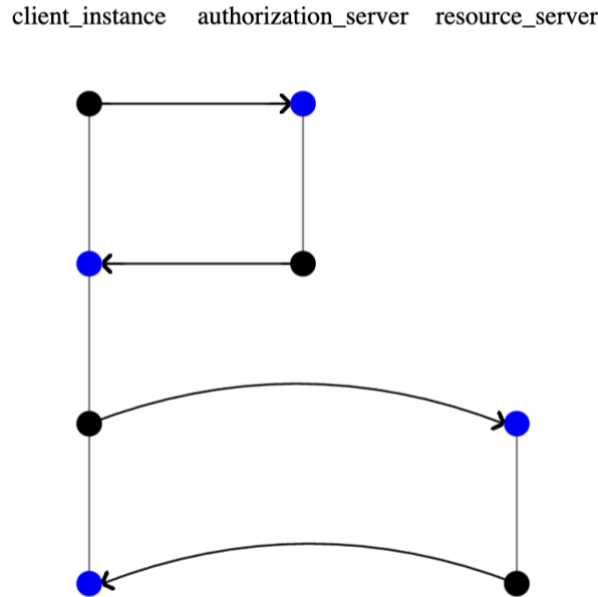


Figure 5: Client Instance Perspective (Software-only Authorization)

After conducting simulations using our model, CPSA generated results from the client-instance perspective that align with our expectations. It is shown in Figure 5. As demonstrated in the previously presented sequence diagram, the client initiates a request to the authorization server. The authorization server then authenticates the client and issues an access token based on the request. Next, the client utilizes this access token to request data from the resource server. After verifying the access token, the resource server returns the requested data to the client.

This ideal scenario aligns with the sequence of execution shown in the earlier slide, and there is no possibility of an alternative sequence that would raise security concerns. The CPSA-generated results confirm the effectiveness of our model in maintaining secure access control practices.

Results - Authorization Server Perspective Shape

authorization_server client_instance

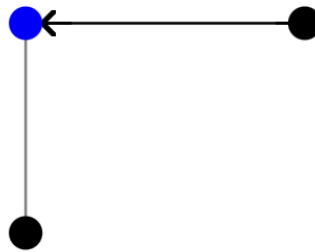


Figure 6: Authorization Server Perspective (Software-only Authorization)

The result generated by CPSA from the authorization server's perspective align with our expectations and is deemed satisfactory. This is shown in Figure 6. When a client makes a request to the authorization server, the server issues an access token in response. It is worth noting that in the CPSA-generated diagram, there is no line connecting the authorization server and the client once the token has been successfully sent to the client. It is important to highlight that this result is solely from the authorization server's viewpoint, meaning that the server cannot determine if the client successfully received the token, even if it was sent successfully.

Results - Resource Server Perspective Shape

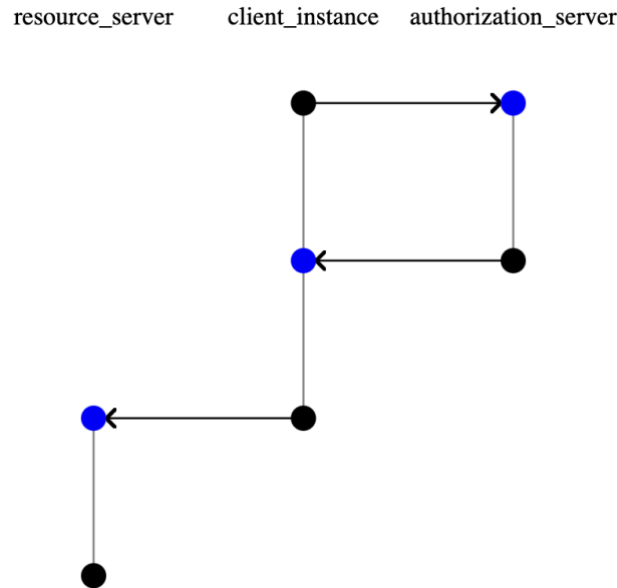


Figure 7: Resource Server Perspective (Software-only Authorization)

The results obtained from the resource server's perspective align with our expectations and follow the primary protocol sequence illustrated in earlier presentations. This is shown in Figure 7. As per the sequence, the client initiates a request to the authorization server, and after authentication, the authorization server issues an access token in response. The client then utilizes this token to make an API request to the resource server. Upon verifying the client's access rights, the resource server sends the requested information back to the client.

These results confirm our analysis of the GNAP protocol's Software-only Authentication sequence. We are pleased to report that there are no evident structural flaws that could jeopardize its security, at least for the model we have developed.

User-Code Interaction Sequence

Let's discuss the second sequence that we modeled, which is the User-code interaction shown in Figure 8. It is worth noting that this sequence can be split into two distinct and independent sequences. In the first sequence, the client can send a continue request to the authorization server before the authentication and authorization process between the authorization server and the end user is complete.

In the second sequence, the client instance sends a continue request to the authorization server only after the authentication and authorization process between the authorization server and the end user is complete.

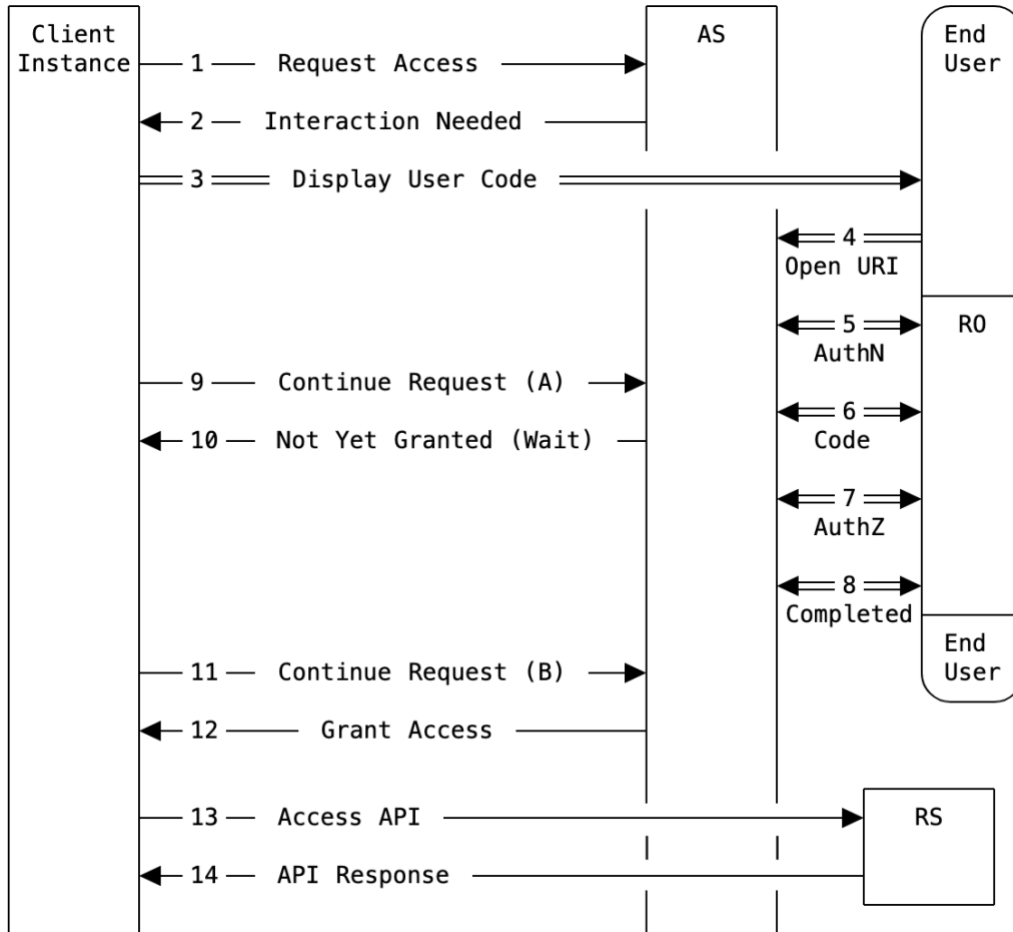


Figure 8: User-Code Interaction (GNAP)

For our modeling in CPSA, we have opted for the second sequence. This involves disregarding interactions 9 and 10, namely the Continue Request (A) and the Not Yet Granted (Wait) interaction. Consequently, our model only accounts for interactions 1 to 8, and 11 to 14. In the chosen sequence, the following steps are followed:

1. The client instance requests access to the resource and indicates its ability to display a user code.

2. The authorization server recognizes the need for interaction and sends a user code to the user. Additionally, the authorization server includes the necessary information for the client instance to continue the request in step 8. This continuation information is associated with an ongoing request and will be referred to in steps 4, 6, and 8.
3. The client instance saves the continuation information received in step 2 for use in step 8. The client instance then communicates the user code to the user as provided by the authorization server in step 2.
4. The user accesses the user code unique resource identifier (URI) using their browser. The URI is stable and can be communicated via client software documentation, AS documentation, or the client software itself. Since it is assumed that the resource owner will use a secondary device to interact with the authorization server, the client instance does not provide a mechanism to launch the resource owner's browser at this URI.
5. The end user authenticates with the authorization server, acting as the resource owner.
6. The resource owner enters the user code received in step 3 to the authorization server. The authorization server validates this code against the current ongoing request.
7. As the resource owner, the user approves the pending request from the client instance.
8. Once the authorization server has finished interacting with the user, it informs the resource owner that the request has been completed.
9. The client instance then begins polling the authorization server with the continuation information received in step 3.
10. If the request has been authorized, the authorization server grants the client instance access to the requested information via access tokens and direct subject information.
11. The client instance uses the access token to call the resource server.
12. The resource server verifies the access token and provides an appropriate response for the API.

Request Access (1)

In this particular interaction, the client instance sends an access request to the authorization server, which includes the following components:

1. The client instance key
2. The resource server ID
3. The resource server permission

The access request is signed by the client instance using its private key, which ensures that the message is tamper-proof.

Interaction Needed (2)

After validating the request from the client instance, the authorization server sends back the following information:

1. User Code: This code is necessary for the end-user to authorize the request.
2. Continue Access Token: This token is unique to the current request and is generated using the public key of the client instance, a unique random value, the resource server ID, and the resource server permission. The entire token is signed by the authorization server using its private key. This process is similar to generating a JSON Web Token (JWT).

Display User Code (3)

In this interaction, the client instance displays the user code to the end-user, who uses it to access the URI necessary to communicate with the authorization server and share the code. However, it is challenging to model this physical interaction in CPSA. To overcome this, we utilized the channel feature in CPSA.

At this stage, we create a new channel between the client instance and the end-user. This channel is configured to be always authentic and confidential. We assumed that this interaction is secure and confidential, but it is possible that someone could see the code in the real world, and the

interaction might not be entirely authentic and confidential in the sense that someone could be nearby and observe the code.

Open URI (4), AuthN (5), Code (6), AuthZ (7) and Completed (8)

To summarize, in order to simulate the Open URI, Authentication AuthN, Code, Authorization AuthZ, and Completed interactions, we had to assume that the end user knows the correct URI to communicate with the authorization server and establish a connection through the channels feature in CPSA. Since GNAP does not specify a particular authentication protocol, we used Long-Term-Keys (LTKs) to represent encryption keys between trusted parties. By passing the user code encrypted using LTKs between the end user and the authorization server, we were able to simulate all five interactions.

Continue Request (B) (11)

At this stage, the client instance sends a "continue" request to the authorization server to inquire about the status of its original request. In order to execute this request, the client instance must include the exact same "continue" access token that it received from the server during the previous "Interaction Required" phase.

Afterward, to ensure secure communication, the client instance signs the "continue" access token with its private key and sends it to the authorization server for verification.

Grant Access (12)

This interaction occurs when the authorization server grants a client instance's request after successfully authenticating and authorizing the end user. During this interaction, the authorization server sends the following:

1. Resource Server Permission: This indicates which resource server(s) the client instance is authorized to access.
2. Final Access Token: This access token is specific to the client instance and is required for making API calls to the authorized resource server(s). To generate this token, the authorization server uses the client instance's public key, a unique random value (to differentiate between

various requests), resource server ID, and resource server permission, all signed with the authorization server's private key for security purposes.

Access API (13)

To access the API, the client instance uses the access token received from the authorization server to make an access request for resources on the authorized resource server. Before sending this request to the resource server, the client instance signs it with its private key to ensure secure communication.

API Response (14)

Similar to the Software-only Authorization flow, the resource owner verifies the access token and confirms that the client instance and the provided token have sufficient permissions to carry out the requested access request. Once validated, the resource owner sends the requested data and a hash of the data and access token to bind this response to the request that sent the access token.

Results – Client Instance Perspective Shape

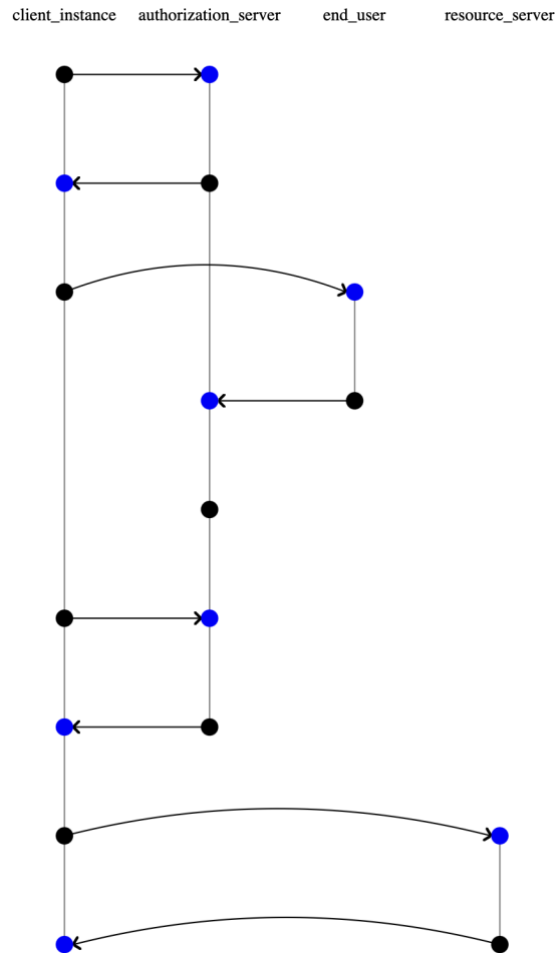


Figure 9: Client Instance Perspective 1 (User-Code Interaction)

After simulating our model, we were able to generate a shape using CPSA from the client instance's perspective. This shape is a clear example of how the authorization process works in software-only authorization.

The shape depicts the client instance initiating the process by sending a request for access to the authorization server. The authorization server responds by sending the user code to be displayed to the end-user. The client instance then shares this code with the end-user.

The end-user authenticates and authorizes the authorization server by sharing the user code. Once this is completed, the client instance sends a continue access request to the authorization server. The authorization server grants access by sharing a token that the client instance uses to make an API call to the resource server. The resource server verifies the request and privileges and then sends the required data back to the client instance.

Results – Client Instance Perspective (Unrealized)

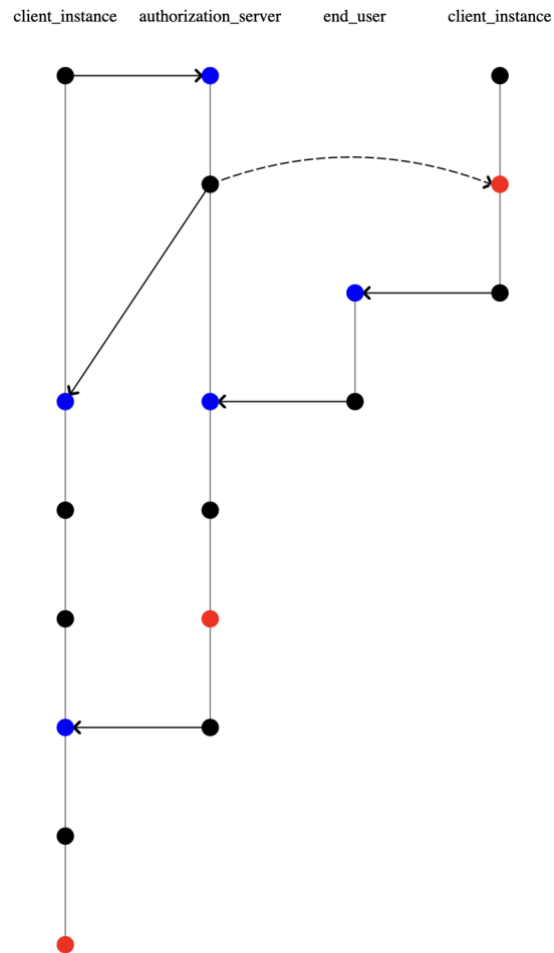


Figure 10: Client Instance Perspective 2 (User-Code Interaction)

We encountered a situation while simulating our model that CPSA failed to recognize. The nodes colored in red in the generated picture indicate the points that CPSA could not comprehend.

However, it is important to note that this situation is not a valid scenario, which means that it does not provide us with any meaningful insights. Therefore, we cannot classify this as a shape, and it does not contribute to our analysis.

Results - Authorization Server Perspective Shape

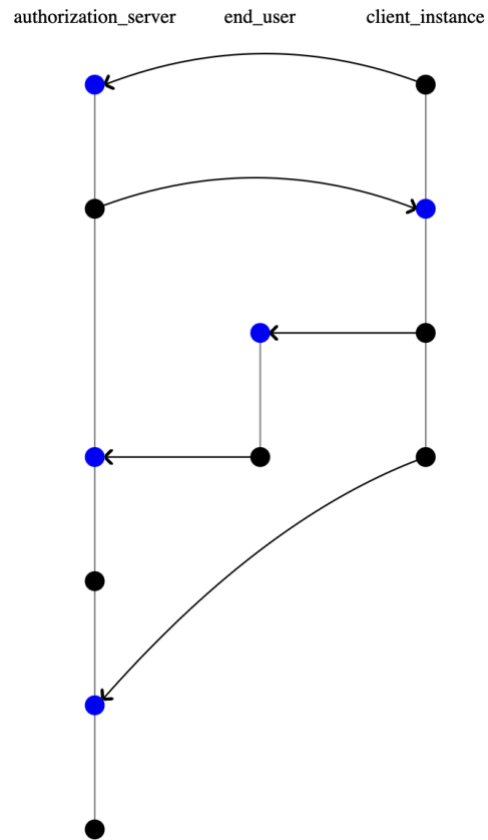


Figure 11: Authorization Server Perspective 1 (User-Code Interaction)

The following graph in Figure 13 displays the interaction from the perspective of the authorization server. This is a satisfactory result as there are no issues with this interaction.

Results - Resource Server Perspective Shape

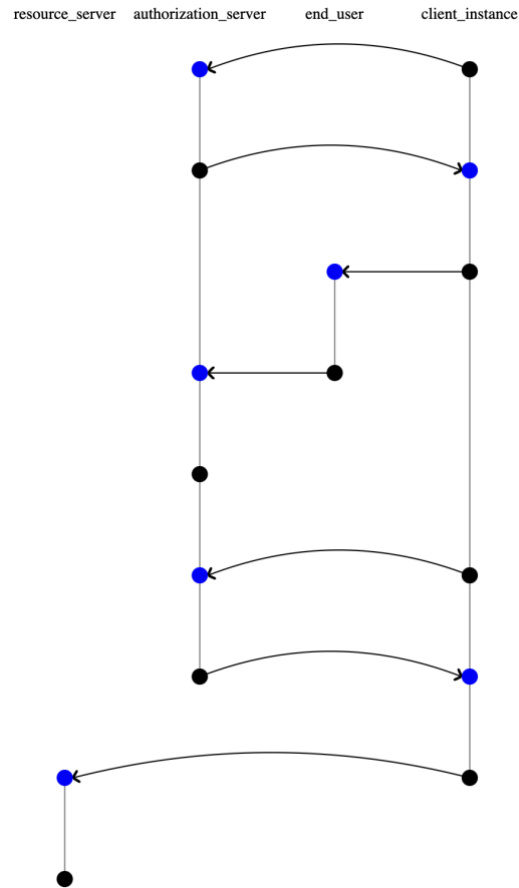


Figure 12: Resource Server Perspective 1 (User-Code Interaction)

Figure 15 shows a legitimate and expected interaction from the resource server's perspective.

Next Steps

The future scope of this project is vast and includes multiple areas of potential improvement.

Firstly, it is essential to analyze the various shapes that were generated during the generation of our existing model. Currently, we have only analyzed a limited set of shapes, and there are many more shapes that need to be analyzed.

Secondly, our current model is prepared using only the second sequence of GNAP, which was discussed earlier. However, there are several other flows of User-Code interaction that can occur, and these need to be modeled as well.

Thirdly, we believe that our existing model can be further improved to generate better shapes. The team working on this project in the future can consider this as an area of focus.

Lastly, it is crucial to model several other GNAP sequences to complete the overall formal analysis of this protocol. The successful modeling of these sequences will add to the completeness of the project and make it a more comprehensive analysis.

Conclusion

To conclude this project, there are two key findings that we want to highlight. Firstly, based on the sequences that we have analyzed, we have not found any vulnerabilities in the GNAP protocol. However, it is important to note that our analysis is limited, and more extensive analysis is required to make a conclusive determination about the security of GNAP. We hope that our results will be valuable in furthering the analysis of this protocol.

Secondly, we have identified several ambiguities in the GNAP documentation. For instance, there is no mention of the protocol used for interactions during certain stages. This ambiguity leaves room for assumptions, and it is possible that analysts may make interpretations that do not align with the actual implementation.

Lastly, we would like to express our gratitude to Ed and Enis for their invaluable support throughout this project. Although the project was initially alien to our team, they helped us to quickly onboard and get up to speed. Their constant support enabled us to achieve all the targets we had set at the outset of the project.

Team Member Biographies

Manasa Kandala

Manasa is a graduate student at the University of Texas at Dallas, where she is studying Computer Science with a focus on Cybersecurity. Before her studies at UTD, she worked as a software engineer for two years, gaining experience in both cloud development and full-stack development. To enhance her knowledge in this field of security, she took up courses such as

Network Security, System Security, Malicious Code Analysis, and Information Security. She is passionate about creating cloud applications that are both efficient and secure.

Pragathi Vemulapalli

As a Computer Science Graduate student specializing in Data Science at The University of Texas at Dallas and as a student assistant working at the Office of Information Technology, Pragathi possesses skills related to detecting vulnerabilities or anomalies and finding a solution to them. She worked on various Cloud migrations, developed full-stack applications and user macros, worked on various scripting languages such as Bash, python, and created network diagrams. She also took courses like Machine Learning, Operating systems, Advanced Mathematics as well as computer networks during her undergraduate studies. With her experience and skills, she can execute this research project and get the desired results.

Rahul Muthyam

Rahul is a graduate student at the University of Texas at Dallas studying Computer Science with a focus on Cybersecurity and has been interested in cybersecurity long before their undergraduate studies. He is also a software engineer by profession and specializes in creating products for the cloud. Through taking courses such as Network Security, System Security, and Information Security, he has gained a thorough understanding of the field. His passion for cybersecurity is demonstrated through various projects, and he is confident that his skills and experience will allow them to become a valuable member of the team.

Saloni Shivdasani

Saloni Shivdasani is a master's student at the University of Texas at Dallas majoring in Computer Science with a concentration in cyber security. She has been in the software development industry for more than 2 years. At her current job at a consulting firm, she has been utilizing skills in project management, cloud administration, continuous integration and delivery on the cloud and full-stack mobile development. She also has had a year of experience as an Engineering Analyst at the Information Security Office at UTD utilizing cloud administration along with Python and shell automation. While at UTD, she had taken classes on information security, data and application security, network security, and language-based security along with database systems, telecommunication systems, advanced operating systems, and software testing.

Outside of university and work, she has taken part in security competitions where she has consistently placed in among the top 10% nationally and received scholarships to attend multiple conferences. She is currently studying to get her security+ credentials.

Shreyans Patel

Shreyans is a graduate student studying Intelligent Systems in Computer Science at The University of Texas at Dallas. He has experience working in the product-based software and technology industry as an Embedded Software Developer. His interest in both hardware and software led him to the world of Linux-based operating systems, which are commonly used for ethical hacking. His undergraduate degree in Electronics and Communication Engineering has provided him with a strong foundation in computer/microprocessor/microcontroller architectures, computer networks and communication, low-level languages, and scripting languages. His undergraduate and graduate education and skills will allow him to provide a useful solution to the problem at hand. He is proficient in C/C++, Python, and Java and knows Machine Learning, Database Design, Operating System Concepts, Data Structures, and Algorithm Analysis and Design.

Bibliography

- [1] A. T. Sherman, E. Lanus, M. Liskov, E. Ziegler, R. Chang, E. Golaszewski, R. Wnuk-Fink, C. J. Bonyadi, M. Yaksetig, and I. Blumenfeld, "Formal Methods Analysis of the Secure Remote Password Protocol," arXiv:2003.07421, 2020.
- [2] [IETF-WG-GNAP] "Grant Negotiation and Authorization Protocol (gnap)," Internet Engineering Task Force (IETF) Working Group GNAP, <https://datatracker.ietf.org/wg/gnap/documents/>
- [3] [IETF-RFC8252] W.Dennis Google and J. Bradley, "OAuth 2.0 for Native Apps," Internet Engineering Task Force (IETF) Request for Comments (RFC) 8252, August 2017, <https://www.rfc-editor.org/rfc/rfc8252>
- [4] Axeland, Åke, and Omar Oueidat. Security Analysis of Attack Surfaces on the Grant Negotiation and Authorization Protocol. Diss. MA thesis. Chalmers University of Technology and

University of Gothenburg, 2021. url: <https://hdl.handle.net/20.500.12380/304105> (cit. on p. 37), 2021.